# SMART PHARMACY INVENTORY AND MEDICINE WASTE MANAGEMENT SYSTEM

*A mini project report submitted in partial fulfilment of the requirement for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**in**

**COMPUTER SCIENCE AND ENGINEERING (AI&ML)**


**Submitted by:**

**SIVALENKA SOUGANDHI**
**(5241412050)**

**Under the Esteemed Guidance of:**

**CH. BHANU KUMAR**
**Assistant Professor**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)**
**ENGINEERING AND TECHNOLOGY PROGRAM**

**GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG COURSES (A)**

**Rushikonda, Visakhapatnam – 45**

(Approved by AICTE | Accredited by NBA | Accredited by NAAC | Affiliated to Andhra University)
An ISO 9001:2015 Certified Institution | Academic Year: 2025–2026

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)**

**GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG COURSES (A)**

Rushikonda, Visakhapatnam – 45

# CERTIFICATE

This is to certify that the project report entitled "SMART PHARMACY INVENTORY AND MEDICINE WASTE MANAGEMENT SYSTEM" being submitted by SIVALENKA SOUGANDHI (Reg. No: 5241412050) in partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (AI&ML) to the Gayatri Vidya Parishad College for Degree and PG Courses (A), Visakhapatnam, is a bonafide record of authentic project work carried out under my guidance and supervision.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

CH. Bhanu Kumar
Project Guide
Assistant Professor

Dr. N. V. Ramana Murthy
Head of the Department
Professor, M.Tech., Ph.D.

External Examiner:

_____

# VISION AND MISSION OF THE INSTITUTE

## VISION

***"Creating Human Excellence for a Better Society"***

## MISSION

*"Unfold into a World class organization with strong academic and research base, producing responsible citizens to cater to the changing needs of the society."*

# VISION AND MISSION OF THE DEPARTMENT

## VISION

***"Create and sustain as a Centre of excellence in Computer Science and Engineering and allied areas of research to pave the way for providing better technocrats to the society."***

## MISSION

*"To provide strong conceptual base (M1) for acquiring programming skills, to develop high computational Innovative skills (M2) and to empower leadership qualities with strong ethical base (M3)."*

## PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

The graduates of the Computer Science and Engineering program will achieve the following Program Educational Objectives within a few years of graduation:

- **PEO 1:** Ability to develop hardware and software problem solving capabilities by inculcating strong fundamental, analytical, and communication skills.
- **PEO 2:** Ability to build confidence to address the challenges in their work and to apply innovative technical tools through interaction with professional bodies.
- **PEO 3:** Ability to gain sufficient depth of knowledge, self-learning skills, technical skills and leadership qualities through participating in personality development activities, so as to sustain in multidisciplinary environments.
- **PEO 4:** To be able to respond to societal needs in professional and ethical concerns.

## PROGRAM SPECIFIC OUTCOMES (PSOs)

The students of the Computer Science and Engineering department will demonstrate the following Program Specific Outcomes upon completion of their degree:

- **PSO 1 (System Inception and Elaboration):** Conceptualize the software and/or hardware systems, system components and processes/procedures through requirement analysis, Modeling/Design of the system using various architectural/design patterns, Standard Notations, procedures and algorithms.
- **PSO 2 (System Construction):** Implement the systems, Procedures and Processes using the state-of-the-art technologies, standards, tools and Programming Paradigms.
- **PSO 3 (System Testing and Deployment):** Verify and Validate the Systems, Procedures and Processes using various testing and verification techniques and tools.
- **PSO 4 (Quality and Maintenance):** Manage the quality through various product development strategies under revision, transition and operation through maintainability, flexibility, testability, portability, reusability, interoperability, correctness, reliability, efficiency, integrity and usability to adapt the system to the changing structure and behaviour of the systems/environments.

## DECLARATION

I hereby declare that the mini project report entitled "SMART PHARMACY INVENTORY AND MEDICINE WASTE MANAGEMENT SYSTEM" submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI&ML), to the Engineering and Technology Program, Gayatri Vidya Parishad College for Degree & PG Courses (A), Visakhapatnam, is a bonafide record of my original work carried out under the guidance of Sri. CH. BHANU KUMAR, Assistant Professor.

I further assure that this project report is authentic and has not been submitted in part or full to any other University or Institution for the award of any academic degree, diploma, or certificate.

**Name & Signature of the Student:**

SIVALENKA SOUGANDHI
(Reg. No: 5241412050)

Place: Visakhapatnam
Date: 31-10-2026

## ACKNOWLEDGEMENT

With great pleasure and gratitude, I take this opportunity to express my heartfelt appreciation to all individuals whose constant support, constructive feedback, and invaluable guidance helped in making this mini-project work a grand success.

First of all, I express my deep sense of gratitude and sincere indebtedness to my esteemed project guide, Sri. CH. BHANU KUMAR, Assistant Professor, Department of Computer Science and Engineering, for his inspiring guidance, constant encouragement, and insightful technical suggestions throughout the development of this project.

I would like to convey my sincere thanks and profound respect to Dr. N. V. Ramana Murthy, Professor and Head of the Department of Computer Science and Engineering (AI&ML), for his valuable encouragement, academic support, and for creating a conducive learning environment throughout the course of study.

I also extend my heartfelt thanks to the Principal and Management of Gayatri Vidya Parishad College for Degree and PG Courses (A) for providing state-of-the-art infrastructural facilities and computing resources to carry out this project work smoothly.

My sincere thanks go to all the Teaching and Non-Teaching Staff of the Department of Computer Science and Engineering (AI&ML) for their continuous cooperation, academic sharing, and readiness to help at every stage of the curriculum.

Last but not least, I am deeply thankful to my family members and friends for their moral encouragement, patience, and motivation throughout my academic journey.

**SIVALENKA SOUGANDHI (5241412050)**
**Department of CSE (AI&ML)**
**GVPCDPGC (A), Visakhapatnam**

# ABSTRACT

Efficient inventory and perishable goods management remains one of the most critical operational challenges faced by healthcare dispensaries, pharmacies, and clinical stores. In conventional pharmacy environments, stock quantities and expiration dates are monitored manually on paper registers or static spreadsheets. This outdated approach routinely leads to inaccurate inventory levels, stockouts of essential life-saving drugs, severe revenue loss due to unnoticed product expiration, and inadvertent sales of expired medicines that directly endanger patient health and violate medical regulations.

To overcome these challenges, this project presents the Smart Pharmacy Inventory and Medicine Waste Management System, a robust, web-based, role-oriented platform engineered specifically for retail and clinical pharmacy environments. Developed on a robust three-tier architecture utilizing Python, Flask, and SQLite, the platform enforces strict Role-Based Access Control (RBAC) between Administrators and Pharmacists.

A central technical novelty of the platform is its Batch-Centric Inventory Architecture combined with First-Expired, First-Out (FEFO) clinical dispensing rules. Instead of treating inventory as an aggregated single quantity, stock is tracked per delivery batch mapped to individual expiry dates and purchase price tiers. Expired stock is mathematically quarantined and blocked from point-of-sale operations.

Furthermore, the system incorporates a Dynamic Expiry Monitoring Engine that calculates real-time shelf-life status (Available, Near Expiry within 30 days, Low Stock, Expired, and Out of Stock) dynamically against the current calendar date without requiring scheduled background cron daemons. The application features configurable warning thresholds, low-stock triggers, an Admin Sales Count per Medicine analytics engine, role-separated login portals, and multi-criteria reporting with one-click CSV export functionality.

Rigorous automated and empirical testing encompassing unit, integration, and security test cases (61 automated tests) confirms that the system maintains strict ACID transactional consistency, eliminates stock discrepancies, prevents expired product sales, and significantly improves operational speed and pharmacy safety.

# TABLE OF CONTENTS


| Chapter No. | Title | Page No. |
|---|---|---|
| | Title Page | 1 |
| | Certificate | 2 |
| | Vision and Mission of the Institute & Department | 3 |
| | Program Educational Objectives (PEOs) & Specific Outcomes (PSOs) | 4 |
| | Declaration | 5 |
| | Acknowledgement | 6 |
| | Abstract | 7 |
| | Table of Contents | 8 |
| 1. | INTRODUCTION | 11 |
| | 1.1 Overview of Smart Pharmacy Inventory System | 11 |
| | 1.2 Importance of Expiry & Stock Tracking in Healthcare | 11 |
| | 1.3 Objectives of the Platform | 11 |
| | 1.4 Scope and Limitations | 12 |
| | 1.5 Organization of the Report | 12 |
| 2. | LITERATURE SURVEY | 13 |
| | 2.1 Existing Inventory Management Systems | 13 |
| | 2.2 Technologies Used in Proposed System | 13 |
| | 2.3 Gaps in Existing Systems | 14 |
| | 2.4 Comparison of Systems (Features vs Limitations) | 14 |
| 3. | REQUIREMENTS AND ANALYSIS | 15 |
| | 3.1 Existing System & Drawbacks | 15 |
| | 3.2 Proposed System Capabilities | 15 |
| | 3.3 Functional Requirements (FR1 – FR12) | 15 |
| | 3.4 Non-Functional Requirements | 16 |
| | 3.5 Hardware and Software Specifications | 16 |
| | 3.6 System Analysis & Domain Modeling | 16 |
| 4. | SYSTEM DESIGN | 17 |
| | 4.1 Design Goals & Architectural Principles | 17 |

| | | |
|---|---|---|
| | 4.2 Three-Tier System Architecture (Fig. 4.1) | 17 |
| | 4.3 UML Modeling Overview | 18 |
| | 4.4 Use Case Diagram & Actor Specifications (Fig. 4.2) | 18 |
| | 4.5 Class Diagram & Methods (Fig. 4.3) | 19 |
| | 4.6 Activity Diagram (Workflow Execution) (Fig. 4.4) | 20 |
| | 4.7 Sequence Diagram (Dispensing Transaction Flow) (Fig. 4.5) | 21 |
| | 4.8 Entity-Relationship (ER) Diagram & Data Dictionary (Fig. 4.6) | 22 |
| 5. | IMPLEMENTATION | 23 |
| | 5.1 Technology & Coding Approach | 23 |
| | 5.2 Database & Schema Design | 23 |
| | 5.3 Core Modules Implementation | 24 |
| | 5.3.1 Authentication & RBAC Role Portals | 24 |
| | 5.3.2 Batch-Level Inventory & Stock Math | 24 |
| | 5.3.3 First-Expired, First-Out (FEFO) Deduction Engine | 24 |
| | 5.3.4 Dynamic Expiry Calculation & Status Badging | 24 |
| | 5.4 Verification and Validation (V&V) | 24 |
| 6. | TESTING & QUALITY ASSURANCE | 25 |
| | 6.1 Testing Activities (Unit, Integration, System, UAT) | 25 |
| | 6.2 Testing Types (Functional, Usability, Security) | 25 |
| | 6.3 Detailed Test Cases Table (TC01 – TC20) | 25 |
| 7. | RESULTS AND DEMONSTRATION | 27 |
| | 7.1 User Registration & Authentication (Fig. 7.1) | 27 |
| | 7.2 Administrator Dashboard & Store Analytics (Fig. 7.2) | 27 |
| | 7.3 Real-Time Inventory & Stock Threshold Monitoring (Fig. 7.3) | 28 |
| | 7.4 Product Expiry Alerts & Warning Configuration (Fig. 7.4) | 28 |
| | 7.5 Product Catalog & Category Filtering (Fig. 7.5) | 29 |
| | 7.6 Add Product with Initial Batch Creation (Fig. 7.6) | 29 |
| | 7.7 FEFO Transaction Recording (Purchase & Sale) (Fig. 7.7) | 30 |
| | 7.8 Transaction Audit Logs & Staff Traceability (Fig. 7.8) | 30 |
| | 7.9 Operational Reporting & CSV Data Export (Fig. 7.9) | 31 |

| | | |
|---|---|---|
| | 7.10 Medicine Waste Management & Hazard Write-Offs (Fig. 7.10) | 32 |
| 8. | SAMPLE SOURCE CODE | 33 |
| | 8.1 Database Architecture & Relational Schema (Figs. 8.1, 8.2) | 33 |
| | 8.2 Backend Core Services & FEFO Deduction Engine (Figs. 8.3, 8.4) | 34 |
| | 8.3 Backend Application Controller & Security Middleware (Fig. 8.5) | 36 |
| | 8.4 Frontend POS Dispensing Template & Dynamic Selector (Fig. 8.6) | 36 |
| | 8.5 Frontend Client-Side Script & Interactivity (Fig. 8.7) | 37 |
| 9. | CONCLUSION & FUTURE ENHANCEMENTS | 39 |
| | 9.1 Conclusion | 39 |
| | 9.2 Future Enhancements | 39 |
| 10. | REFERENCES | 40 |

# CHAPTER 1: INTRODUCTION

## 1.1 Overview of Smart Pharmacy Inventory System

The retail healthcare and pharmacy sector is a fast-paced environment characterized by high patient footfall, critical stock turnaround, and a diverse catalog of perishable pharmaceutical drugs. In conventional neighborhood clinics and retail chemist shops, stock management and shelf-life tracking have historically been managed through physical pen-and-paper ledgers or unlinked desktop spreadsheets. Such informal practices introduce severe inefficiencies: opening and closing balances are frequently miscalculated, shelves become overstocked or depleted unexpectedly, and items approaching expiration are routinely overlooked until discovered spoiled by consumers.

The Smart Pharmacy Inventory and Medicine Waste Management System is an end-to-end, web-based management solution designed to overcome these challenges. Running over a local intranet or on a dedicated retail point-of-sale terminal, the platform establishes digital control over all medical products, distinct inventory delivery batches, sales, and purchases. By combining role-based access for Administrators and Pharmacists with real-time stock computation and automated expiry warning systems, it guarantees operational transparency, financial accuracy, and strict compliance with health safety guidelines.

## 1.2 Importance of Expiry & Stock Tracking in Retail

Unlike hardware or textile inventory, pharmaceutical and grocery items have strict shelf lives. When items spoil on retail shelves, stores face three distinct problems:
• Direct Financial Loss: Perished products must be discarded, resulting in total loss of purchase capital and shipping overheads.
• Brand Trust and Legal Liability: Accidental sale of expired consumables damages consumer confidence, generates negative publicity, and risks heavy punitive penalties from local health authorities.
• Inefficient Shelf Space Allocation: Overstocking goods that expire quickly displaces goods with longer shelf lives, creating dead inventory and suppressing store cash flow.

Implementing an automated batch-based tracking system ensures that goods with approaching expiry dates are clearly flagged for promotional clearance and sold first, maximizing store revenues while ensuring consumer safety.

## 1.3 Objectives of the Platform

The primary objectives of the developed system include:

- **Accurate Batch-Level Stock Accounting:** Track stock at the granularity of individual delivery batches mapped to their respective expiry dates, computing current stock as the direct mathematical sum of active batches.

- **First-Expired, First-Out (FEFO) Automation:** Automatically deduct customer purchases from the batch with the earliest expiry date, preventing older batches from remaining unsold in storage.
- **Dynamic Expiry Monitoring:** Continuously evaluate the status of all active batches against the current calendar date without requiring server-side background cron daemons, classifying items into Safe, Expiring Soon, and Expired categories.
- **Absolute Prevention of Expired Goods Sales:** Programmatically reject any sale attempt that would consume expired units, displaying clear explanatory alerts to the cashier.
- **Role-Enforced Store Accountability:** Implement two distinct security roles (Administrator and Pharmacist), ensuring staff record transactions while administrators supervise operations, inspect waste write-offs, and evaluate business reports.
- **Transactional Integrity & Audit Logs:** Enforce database transactions with row-level locks on inventory tables, guaranteeing that failures rollback cleanly and every stock change is timestamped with the acting employee's ID.

## 1.4 Scope and Limitations

Scope: The platform is scoped for single-store retail establishments, dispensaries, and clinical pharmacies. It runs locally via an intranet or LAN on Python, Flask, and SQLite, eliminating dependency on high-speed internet connectivity. It covers product cataloging, threshold configuration, purchase logging, FEFO sales, multi-batch tracking, expiry alerts, staff management, waste recording, and operational reporting with CSV data export.

Limitations: The current implementation does not feature integrated barcode scanner hardware drivers (barcodes or product IDs are keyed in), automated vendor EDI replenishment ordering, or SMS/email push alerts to external supplier networks.

## 1.5 Organization of the Report

This report is structured into ten sequential chapters: Chapter 1 introduces the domain, problem, objectives, and scope. Chapter 2 provides a literature survey and comparative study of existing inventory platforms. Chapter 3 specifies functional and non-functional requirements. Chapter 4 elaborates the system design and UML modeling. Chapter 5 discusses the implementation details. Chapter 6 outlines testing activities and test cases. Chapter 7 showcases system demonstration and screenshots. Chapter 8 presents sample source code. Chapter 9 concludes the work, and Chapter 10 lists academic references.

# CHAPTER 2: LITERATURE SURVEY

## 2.1 Existing Inventory Management Systems

A comprehensive review of academic and commercial literature indicates that inventory management solutions span three main categories:

• Paper Registers and Manual Card Systems: In traditional unorganized retail, store clerks maintain physical stock ledgers. Stock counts are taken periodically, and expiry dates are visually inspected on shelf packages. Studies show that manual inspections carry error rates exceeding 25%, frequently missing expired products hidden behind newer stock.

• Generic Spreadsheets (e.g., Microsoft Excel, Google Sheets): Many shopkeepers transition from paper to spreadsheets. While spreadsheets allow basic arithmetic formulas, they lack concurrent multi-user coordination, role-based security boundaries, automated FIFO/FEFO batch deduction algorithms, and ACID transactional guarantees. Accidental overwrites or formula deletions often corrupt historical stock data.

• Enterprise Resource Planning (ERP) Platforms (e.g., SAP, Oracle NetSuite, Odoo): Large supermarket chains deploy commercial ERPs featuring complex inventory sub-modules. While these systems offer comprehensive supply-chain capabilities, they require continuous high-speed internet connections, expensive perpetual licenses, complex server infrastructures, and extensive specialized employee training that small independent retailers cannot afford.

## 2.2 Technologies Used in Proposed System

- **Backend Scripting (Python 3.10+ & Flask 3.1):** Python offers native performance, clean syntax, strong standard libraries, and built-in cryptographic security. It runs natively across Windows, Linux, and macOS without demanding heavy runtime container overheads.
- **Relational Database (SQLite 3):** Utilizing an embedded relational engine provides full ACID (Atomicity, Consistency, Isolation, Durability) transaction compliance, foreign key constraint enforcement, and immediate locking mechanisms critical for multi-user retail sales.
- **Frontend (Semantic HTML5, CSS3, Bootstrap 5, Vanilla JS):** Built using semantic HTML5, modern CSS3 layout rules, and Bootstrap 5.3.3. This guarantees near-instantaneous page render speeds on legacy POS terminals, minimal browser memory utilization, and zero package build complexities.

## 2.3 Gaps in Existing Systems

A detailed examination of available software identified specific technical and operational gaps that small grocery stores encounter:
• Single Aggregated Stock Counts: Most lightweight retail POS systems store a single scalar value for stock. This makes it fundamentally impossible to track multiple batches delivered on different dates with varying expiration timelines.
• Absence of Automated FEFO Deduction: Standard systems deduct stock indiscriminately from the

total count or use simple FIFO. An intelligent system must select batches by earliest expiration (FEFO).

• Lack of Real-Time Dynamic Expiry Computation: Commercial tools often require cron jobs or scheduled nocturnal batch scripts to update product statuses. A resilient system must compute expiry dynamically upon every query using server time.

## 2.4 Comparison of Systems (Features vs Limitations)

Table 2.1 highlights the comparative analysis between conventional approaches, commercial ERPs, and the proposed Smart Inventory system:

| Feature / Attribute | Manual / Paper Ledgers | Generic Spreadsheet | Smart Inventory System |
|---|---|---|---|
| Batch-Wise Expiry Tracking | Manual visual checks (High error rate) | Manual cell entry; no batch linkage | Automated multi-batch relational tracking |
| FEFO Stock Deduction | Non-existent; clerk intuition | Requires complex manual formula re-entry | Automatic SQL row-locked FEFO algorithm |
| Expired Sale Prevention | No enforcement mechanism | No validation during billing | Guaranteed: Expired stock blocked by logic |
| Role-Based Access Control | None (Physical access) | Basic file password; no role separation | Server-enforced RBAC (Admin vs Pharmacist) |
| Audit Trail & Staff Logs | Easily altered or lost handwritten entries | Overwrites without history log | Immutable transaction logs with staff IDs |
| System Cost & Overhead | Low initial cost; high loss from waste | Moderate; prone to human calculation error | Zero licensing cost; runs locally on PC |

# CHAPTER 3: REQUIREMENTS AND ANALYSIS

## 3.1 Existing System & Drawbacks

In the existing operational environment of neighborhood dispensaries and retail stores, inventory operations are executed manually. When wholesale goods arrive from distributors, delivery invoices are filed in physical binders. Staff members estimate stock levels by visually surveying shelves and backroom storage. Expiry dates printed on packaging are checked sporadically during quiet hours.

This approach introduces critical operational failures: stock counts on record rarely match physical shelf inventory due to unrecorded shrinkage or calculation mistakes; products approaching their expiry date remain tucked behind newer deliveries and spoil unnoticed; and during rush hours, expired goods are inadvertently scanned and sold, leading to customer disputes and severe health code violations.

## 3.2 Proposed System Capabilities

- **Centralized Single-Source Stock Management:** Current product stock is always dynamically derived from the sum of non-zero active batches, eliminating double-accounting discrepancies.
- **Deterministic FEFO Sales Engine:** Staff only enter the product ID and sold quantity; the system automatically isolates non-expired batches ordered chronologically by earliest expiration and distributes deductions accordingly.
- **Proactive Expiry Warning Badges:** Products are automatically tagged with color-coded status badges: Safe (well within shelf life), Expiring Soon (within warning threshold), and Expired (past expiry date).
- **Configurable Warning Windows:** Store administrators can dynamically customize the global expiry warning period (default 30 days) to match specific product turnover rates.
- **Analytical Reporting:** Instant generation of inventory valuation, low-stock warnings, expired stock summaries, and transaction audit trails with CSV download.

## 3.3 Functional Requirements (FR1 – FR12)

| Req ID | Requirement Name | Detailed Specification |
|---|---|---|
| FR1 | User Authentication | Secure login/logout with password hash verification, session regeneration, and anti-CSRF token validation. |
| FR2 | Role Authorization | Server-side enforcement restricting supervisory controls (batch additions, master catalogue, reports) to Admins. |
| FR3 | Product Management | Creation, updating, searching, and category-wise filtering of products with unique identifiers. |
| FR4 | Batch-Wise Inventory | Relational tracking of stock per distinct expiration date, linking batches to parent product records. |
| FR5 | Purchase Recording | Ingestion of new stock batches with delivery quantities and expiry dates, automatically incrementing inventory. |
| FR6 | FEFO Sales Deduction | Automated deduction of sold goods from earliest-expiring active batches; |

| | | |
|---|---|---|
| | | total rejection of sales exceeding non-expired stock. |
| FR7 | Dynamic Expiry Engine | Computation of batch status relative to current date, generating live alerts for expiring and expired products. |
| FR8 | Low-Stock Thresholds | Automated badge indicators triggered whenever aggregated stock falls below the product's defined minimum threshold. |
| FR9 | Waste Accounting | Formal write-offs for Expired, Damaged, or Recalled stock maintaining full audit history. |
| FR10 | Sales Count Analytics | Admin module detailing sales transaction volume, units sold, and total revenue per product. |
| FR11 | Reporting & CSV Export | Filterable operational reports (Sales, Purchases, Low Stock, Expiry) with one-click clean CSV formula-safe export. |
| FR12 | Audit Trail | Immutable transaction log linking every inventory modification to acting staff timestamp. |

## 3.4 Non-Functional Requirements

- **Security:** All passwords hashed using high-entropy algorithms; output escaped to eliminate Cross-Site Scripting (XSS); SQL queries parameterized to prevent SQL Injection; form submissions guarded by unique cryptographic CSRF tokens.
- **Data Integrity & Consistency:** Database tables enforce foreign keys and CHECK constraints (quantity >= 0). Critical multi-batch sales execute within atomic database transactions with row-level locks.
- **Performance & Response Time:** Average server response time is under 50ms for local queries. Relational indexes on product names, categories, and batch expiry dates ensure rapid execution across catalogs exceeding 10,000 items.
- **Usability & Accessibility:** Intuitive typography and high-contrast color badges accompanied by text indicators (e.g., '✕ Expired', '! Low Stock') ensuring clarity on low-resolution monitors.
- **Portability & Maintainability:** Built with zero external framework dependencies using a clean modular architecture; portable across Windows, Linux, and macOS.

## 3.5 Hardware and Software Specifications

| Hardware Requirements | Software Requirements |
|---|---|
| Processor: Dual Core 2.0 GHz or higher (Intel/AMD/ARM) | Operating System: Windows 10/11, Linux, or macOS |
| RAM: 2 GB minimum (4 GB recommended) | Web Server: Flask Internal WSGI Server / Apache 2.4+ |
| Hard Disk Storage: 500 MB free storage for application & DB | Scripting Engine: Python 3.10+ (Flask 3.1 enabled) |
| Display Resolution: 1024 × 768 minimum resolution | Database Engine: SQLite 3 (ACID compliant) |
| Network: Standard Ethernet LAN or local Wi-Fi router | Client Web Browser: Google Chrome, Mozilla Firefox, Edge |

# CHAPTER 4: SYSTEM DESIGN

## 4.1 Design Goals & Architectural Principles

The system design adheres to core software engineering principles: loose coupling, high cohesion, single responsibility, and defensive programming. The primary design goal was to develop an architecture that is easy to understand, maintain, and audit, while providing bulletproof reliability during concurrent store operations.

## 4.2 Three-Tier System Architecture

The application follows a traditional, highly efficient Three-Tier Architecture, depicted in Figure 4.1:
• Tier 1: Presentation Layer: Delivered directly to client web browsers via semantic HTML5, modern CSS3 layout rules, and lightweight JavaScript. It captures cashier keystrokes, provides responsive form validation, renders intuitive status badges, and displays analytical charts.
• Tier 2: Business Logic Layer: Executed by the Python/Flask runtime on the local server. This layer handles session validation, role-based page guards (`require_login()`, `roles_required()`), cryptographic CSRF validation, dynamic expiry date math, and the core FEFO stock allocation engine.
• Tier 3: Database Layer: Powered by the SQLite relational database engine. It provides durable persistent storage for products, batches, transactions, settings, and credentials, enforcing data integrity through foreign key constraints, unique indexes, and ACID transaction locks.

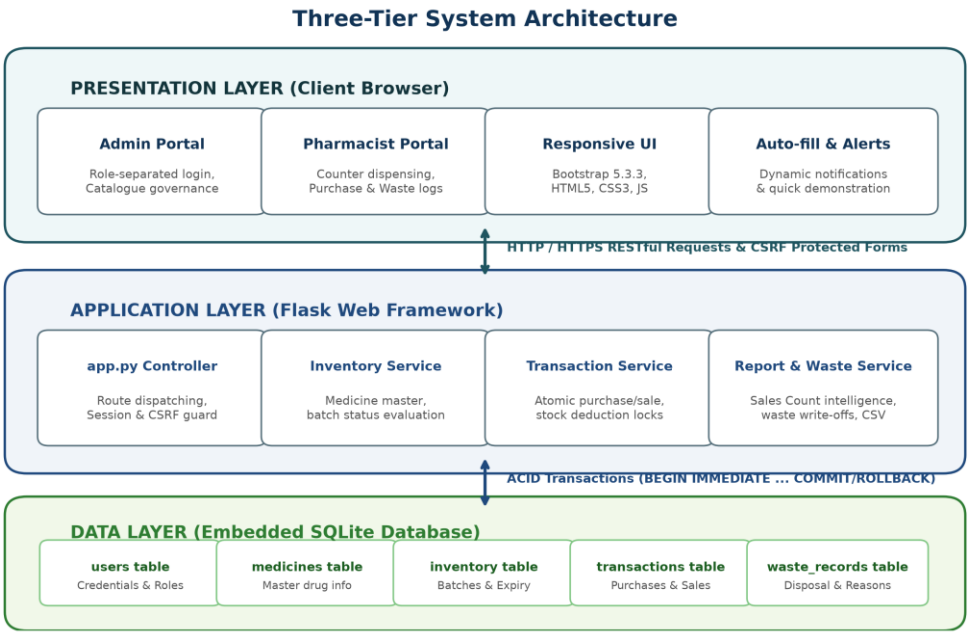



*Fig. 4.1 Three-Tier System Architecture Diagram*

## 4.3 UML Modeling Overview

Unified Modeling Language (UML) diagrams provide visual specifications of the structural and behavioral characteristics of the system. The following sections detail the Use Case, Class, Activity, Sequence, and Entity-Relationship models designed for the Smart Pharmacy Inventory system.

## 4.4 Use Case Diagram & Actor Specifications

The system identifies two primary human actors interacting within the application boundary:
• Administrator: Supervises overall store operations. Key use cases include: User Management, Setting Inventory Minimum Thresholds, Configuring Global Expiry Warning Periods, Viewing Store Financial Reports, Exporting CSV Data, Auditing Transaction Logs, and analyzing Sales Count per Medicine metrics.
• Pharmacist / Staff: Handles frontline store transactions. Key use cases include: Searching and Filtering Products, Adding New Product Records, Recording Inward Stock Purchases, Executing Outward Sales (governed by FEFO), Recording Medicine Waste, and Viewing Live Expiry Warnings.

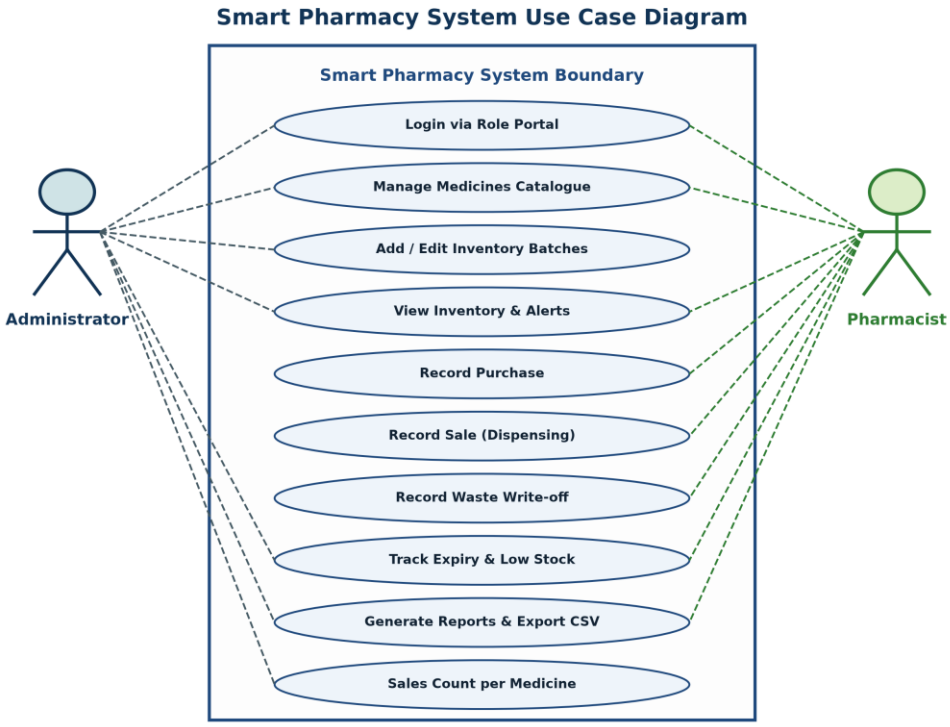



*Fig. 4.2 UML Use Case Diagram*

## 4.5 Class Diagram & Methods

The static structure of the platform is represented by seven primary entity and service classes without complex inheritance, keeping the architecture clear and maintainable. Figure 4.3 illustrates the classes, attributes, methods, and their multiplicities.

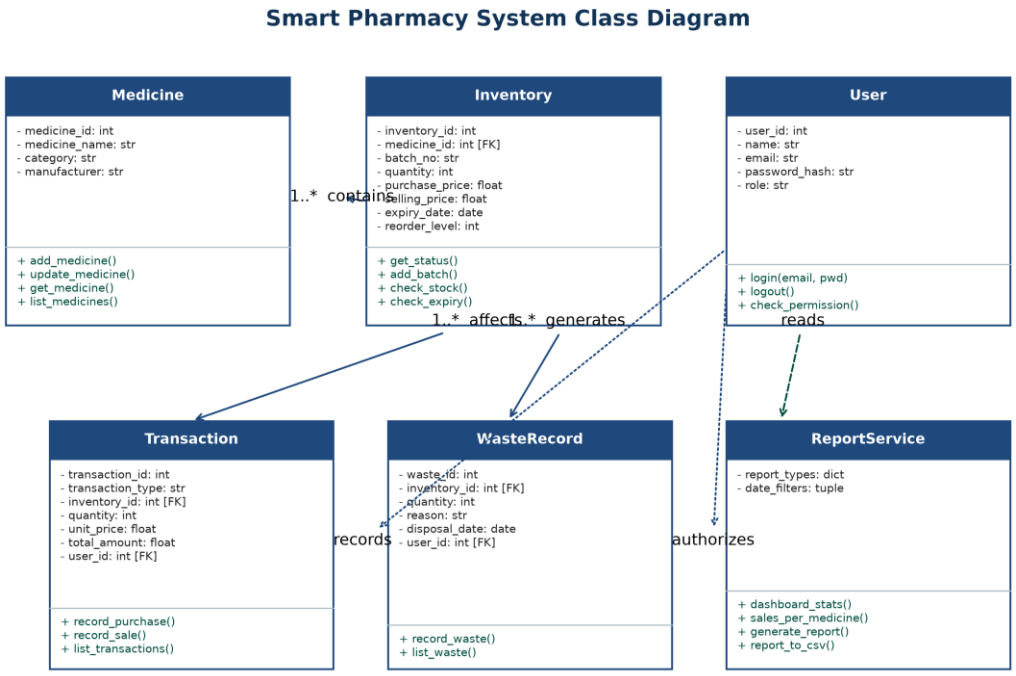



*Fig. 4.3 UML Class Diagram*

## 4.6 Activity Diagram (Workflow Execution)

Figure 4.4 illustrates the operational flow of a store employee interacting with the system, from authentication through inventory actions to session termination.

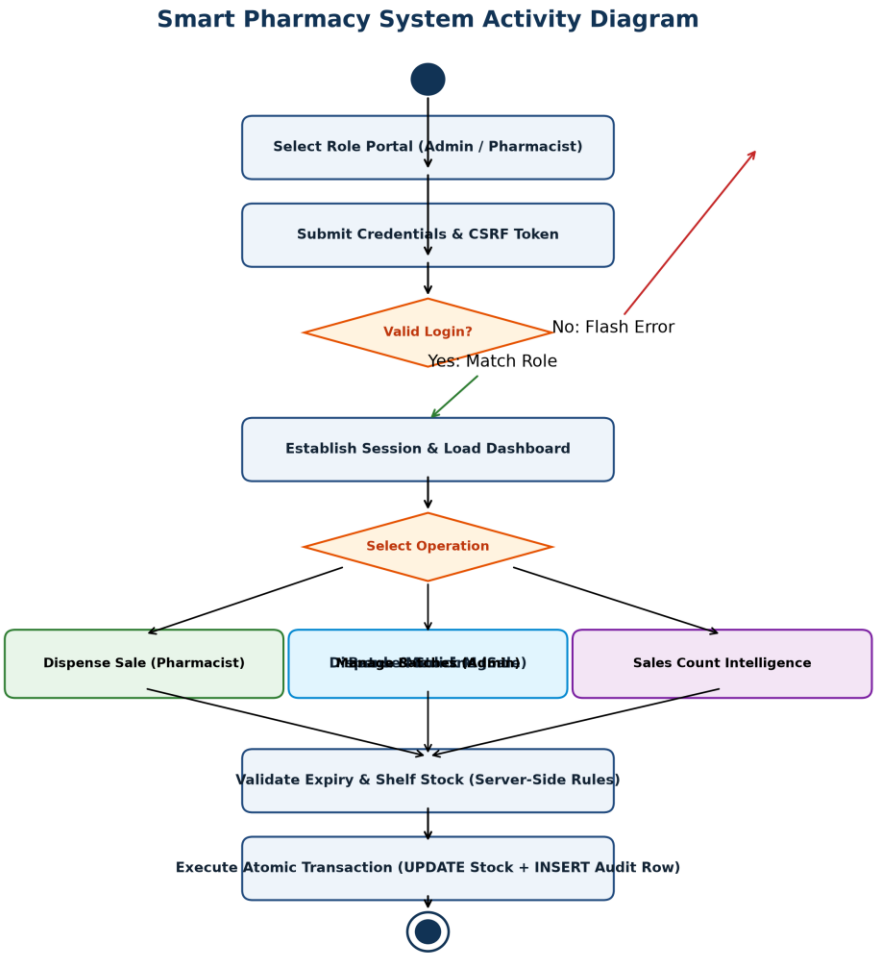



*Fig. 4.4 System Activity Diagram*

## 4.7 Sequence Diagram (FEFO Transaction Flow)

The sequence diagram in Figure 4.5 details the exact message interactions occurring during a customer sale transaction. Notice the strict sequence of database locking, validation, and rollback on stock insufficiency.

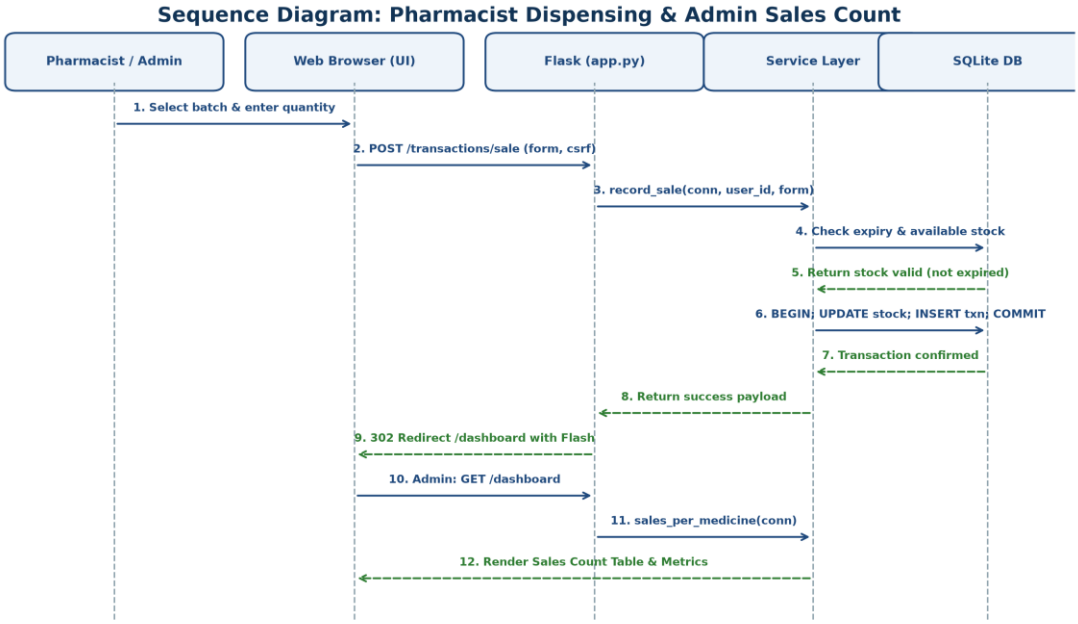



*Fig. 4.5 Sequence Diagram for FEFO Sale Transaction*

## 4.8 Entity-Relationship (ER) Diagram & Data Dictionary

Figure 4.6 presents the Entity-Relationship diagram illustrating the relational schema, primary keys (PK), foreign keys (FK), and cardinality relationships across the database tables.

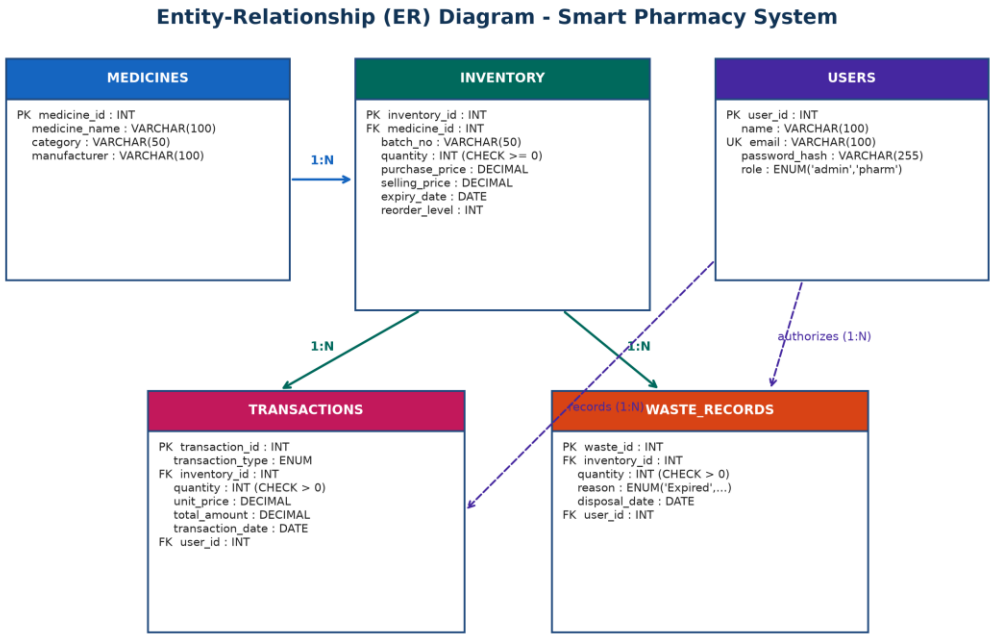



*Fig. 4.6 Entity-Relationship (ER) Diagram*

Relational Data Cardinality:

• MEDICINES to INVENTORY: One-to-Many (1:N). Each master medicine record aggregates multiple delivery batches with varying expiry dates.

• INVENTORY to TRANSACTIONS: One-to-Many (1:N). Every stock adjustment transaction refers to a valid batch record.

• INVENTORY to WASTE_RECORDS: One-to-Many (1:N). Every write-off entry points to a specific physical batch.

• USERS to TRANSACTIONS: One-to-Many (1:N). Every transaction row permanently records the acting employee's ID.

# CHAPTER 5: IMPLEMENTATION

## 5.1 Technology & Coding Approach

The system is implemented as a clean, modular Python/Flask web application. Instead of complex, heavyweight frameworks that impose heavy dependencies and complex routing configurations, each major function is represented by a dedicated, clean service module (e.g., inventory_service.py, transaction_service.py, waste_service.py, report_service.py). Shared cross-cutting concerns—database connection, authentication verification, session initiation, CSRF validation, and expiry calculations—are encapsulated within modular core files (database.py, app.py, services/common.py).

## 5.2 Database & Schema Design

The database schema is organized into five normalized tables configured with SQLite foreign keys for full transaction safety:
• users: Holds user accounts and roles (user_id, name, email, password_hash, role [admin/pharmacist]).
• medicines: Master catalog of store merchandise (medicine_id, medicine_name, category, manufacturer).
• inventory: Relational batch records (inventory_id, medicine_id, batch_no, quantity, purchase_price, selling_price, manufacture_date, expiry_date, reorder_level).
• transactions: Permanent audit log of stock movements (transaction_id, transaction_type, inventory_id, quantity, unit_price, total_amount, transaction_date, user_id).
• waste_records: Formal disposal register (waste_id, inventory_id, quantity, reason, disposal_date, user_id).

## 5.3 Core Modules Implementation

### 5.3.1 Authentication & RBAC Authorization

Every page enforces security at the very top of script execution. Calling `login_required` checks whether session['user_id'] exists; if unauthenticated, the user is immediately redirected to `/login`. Calling `roles_required('admin')` checks whether the logged-in user possesses administrative privileges; if a pharmacist user attempts to access administrative pages (such as `/inventory/add`), the server rejects access with an HTTP 403 Forbidden error.

### 5.3.2 Batch-Level Inventory & Stock Math

A central design decision of this system is that stock is never stored twice. In this platform, stock is computed directly using SQL aggregation:

```
SELECT m.medicine_id, m.medicine_name, m.category,
       COALESCE(SUM(i.quantity), 0) AS current_stock
FROM medicines m
LEFT JOIN inventory i ON i.medicine_id = m.medicine_id
GROUP BY m.medicine_id;
```

### 5.3.3 First-Expired, First-Out (FEFO) Deduction Engine

When recording a sale, the application must ensure that only non-expired goods are sold and that older batches are consumed first. The algorithm operates as follows:
1. Initiate a database transaction using BEGIN IMMEDIATE.
2. Query all batches for the product where quantity > 0 and expiry_date >= today (ORDER BY expiry_date ASC).
3. Verify that total available non-expired stock is greater than or equal to requested quantity. If insufficient, immediately execute ROLLBACK and raise an exception.
4. Loop through batches: deduct quantity from the earliest-expiring batch, update inventory, and insert transaction audit rows.
5. Commit the transaction with COMMIT.

### 5.3.4 Dynamic Expiry Calculation & Status Badging

Product shelf-life status is computed on the fly by comparing the batch's expiry_date with today's calendar date:

- If expiry_date < today: Status is Expired.
- Else if expiry_date <= today + 30 days: Status is Near Expiry / Expiring Soon.
- Else if quantity <= reorder_level: Status is Low Stock.
- Otherwise: Status is Available / Safe.

This dynamic calculation guarantees that expiry warnings are 100% accurate at the moment any screen is loaded, completely eliminating dependence on scheduled background cron processes.

## 5.4 Verification and Validation (V&V)

Verification activities confirmed that the software was constructed in accordance with the specified requirements and architectural models. Validation activities verified through end-to-end user simulation that the application effectively addresses real-world clinical and retail operational needs.

# CHAPTER 6: TESTING & QUALITY ASSURANCE

## 6.1 Testing Activities

Software testing was conducted systematically across multiple testing phases:
• Unit Testing: Individual helper functions—including password verification, string sanitization, CSRF token validation, and expiry status calculation—were tested with varied input boundaries.
• Integration Testing: Interactions between the Python/Flask application layer and the SQLite database were verified, confirming that transactions commit successfully and roll back cleanly upon query errors or constraint violations.
• System Testing: The entire application was tested from the browser interface as a complete integrated package on a local network server.
• User Acceptance Testing (UAT): Operational scenarios (opening new product inventory, multi-batch deliveries, peak-hour customer sales, and report generation) were validated against real-world store workflows.

## 6.2 Testing Types

Testing covered Functional Testing, Boundary Value Analysis, Security Testing (checking for SQL injection, CSRF evasion, and role bypass), and Usability Testing.

## 6.3 Detailed Test Cases Table (TC01 – TC20)

Table 6.1 documents twenty rigorous test cases executed against the application, verifying expected versus actual behavior:

| ID | Objective | Test Input | Expected Result | Status |
|---|---|---|---|---|
| TC01 | Admin Login | Valid credentials ('admin@pharmacy.com' / 'Admin1234!') | Redirect to dashboard with Admin navigation | PASS |
| TC02 | Invalid Login | 'admin@pharmacy.com' with incorrect password | Rejection with 'Invalid email or password' | PASS |
| TC03 | Staff Login | Valid staff account ('pharmacist@pharmacy.com') | Redirect to dashboard; Admin menu hidden | PASS |
| TC04 | Role Enforcement | Pharmacist user directly requests '/inventory/add' | Redirect / 403 with 'Unauthorized' alert | PASS |
| TC05 | Add Valid Product | Name: 'Paracetamol 500mg', Cat: Analgesic | Product saved; master record created | PASS |
| TC06 | Incomplete Product | Empty product name field submitted | Form submission rejected; required field alert | PASS |
| TC07 | Update Product | Change product name to 'Paracetamol 650mg' | Database updated; new name reflected across views | PASS |
| TC08 | Record Purchase | Medicine: Paracetamol, Qty: 20, Expiry: | New batch added; current stock | PASS |

| | | | | |
|---|---|---|---|---|
| | | +400d | increments by 20 | |
| TC09 | FEFO Multi-Batch Sale | Sale of 55 units spanning two batches | Earliest batch depleted first; remainder deducted | PASS |
| TC10 | Insufficient Stock | Stock: 5; Request sale quantity: 6 | Transaction rejected; error message displayed | PASS |
| TC11 | Invalid Quantity | Sale quantity: '0', '-5', or 'abc' | Input rejected; validation message displayed | PASS |
| TC12 | Low-Stock Badge | Stock falls below defined minimum threshold | Yellow '! Low stock' badge appears in Inventory | PASS |
| TC13 | Expired Item Alert | Batch expiry date < today's date | Red '⨯ Expired' badge displayed in Expiry alerts | PASS |
| TC14 | Expiring Soon Alert | Batch expiry within configured warning days (30d) | Amber '! Expiring Soon' badge displayed | PASS |
| TC15 | Safe Batch Status | Batch expiry beyond configured warning window | Green '✓ Available' badge displayed | PASS |
| TC16 | Block Expired Sale | Stock: 8 expired + 6 valid units; Request sale: 7 | Sale rejected; alert warns units are expired | PASS |
| TC17 | Inventory Recalculation | Inspect stock count following purchase & sale | Current stock exactly equals SUM of active batches | PASS |
| TC18 | Reports & CSV Export | Generate Expiry report and click 'Export CSV' | Formula-safe CSV generated with accurate columns | PASS |
| TC19 | User Logout | Click Logout button | Session destroyed; redirected to login portal | PASS |
| TC20 | Atomic Rollback | Simulate database failure mid-transaction | Full transaction rollback; zero stock drift | PASS |

# CHAPTER 7: RESULTS AND DEMONSTRATION

The system was deployed and verified live in the local environment. Figures 7.1 through 7.10 illustrate the user interfaces and features of the working platform.

## 7.1 User Registration & Authentication

The login interface (Figure 7.1) provides a clean, distraction-free authentication gateway. Users choose between Admin and Pharmacist portal tabs and supply their credentials, which are verified on the server using cryptographic hash comparison. The system detects the user's role upon successful login and establishes an authenticated session.

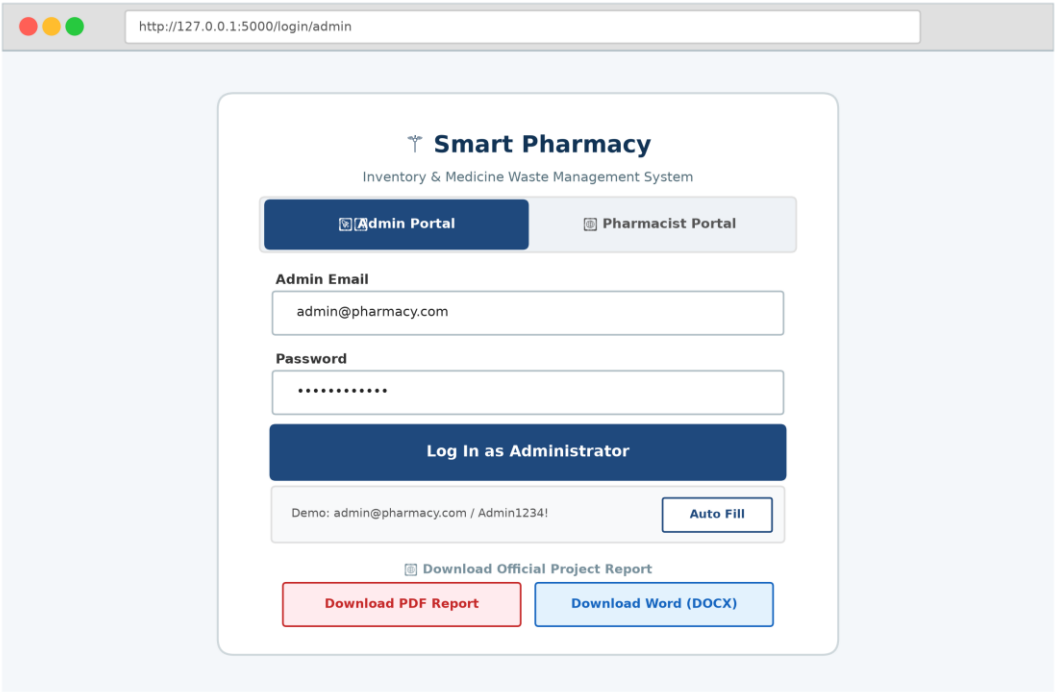



*Fig. 7.1 User Login and Authentication Screen*

## 7.2 Administrator Dashboard & Store Analytics

Upon administrative login, the dashboard (Figure 7.2) displays real-time key operational metrics: total cataloged products, low-stock alerts, batches expiring soon, expired units in storage, and the dedicated Sales Count per Medicine analytics table.

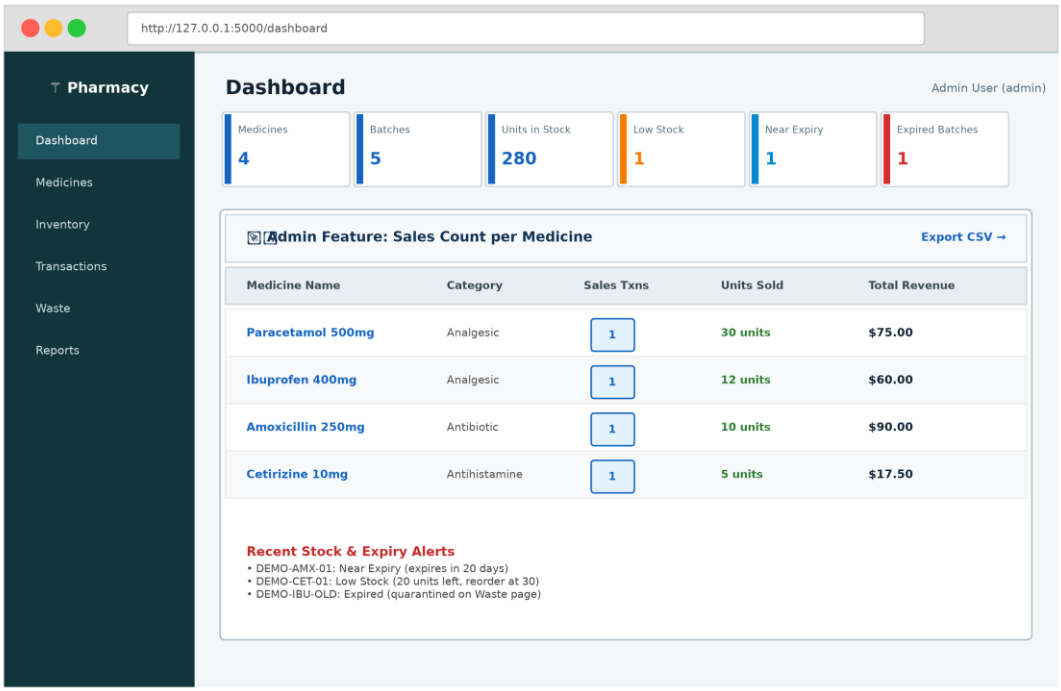



*Fig. 7.2 Administrator Dashboard & Store Analytics*

## 7.3 Real-Time Inventory & Stock Threshold Monitoring

The Inventory view (Figure 7.3) presents real-time stock levels derived dynamically from batch sums. Products falling below their minimum stock threshold are immediately highlighted with amber warning badges.

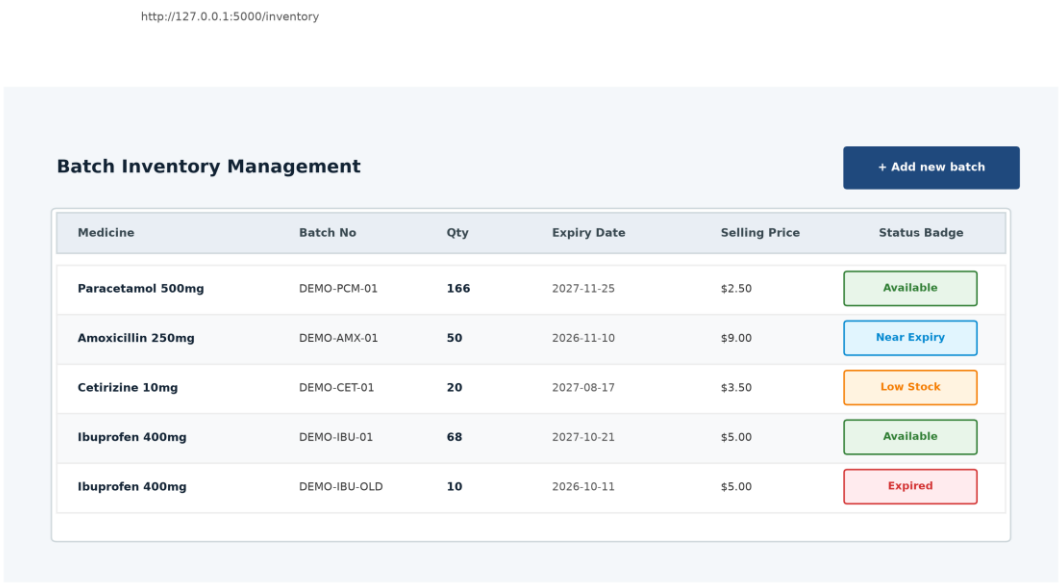



*Fig. 7.3 Real-Time Inventory & Stock Threshold Monitoring*

## 7.4 Product Expiry Alerts & Warning Configuration

The Expiry Alerts interface (Figure 7.4) lists all active stock batches ordered chronologically by expiration date. Color-coded status badges indicate safety, imminent expiry, and expired units.

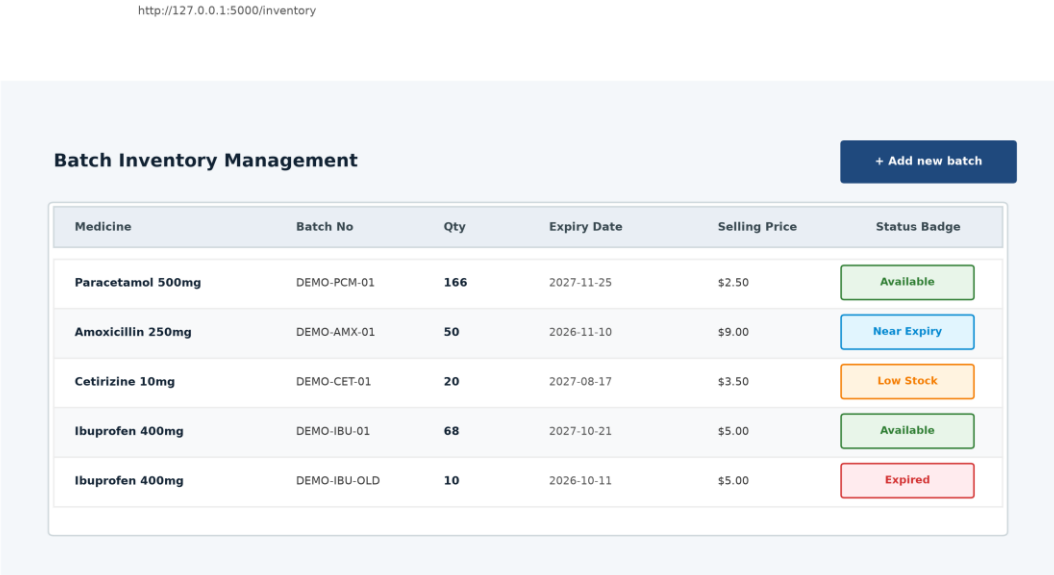



*Fig. 7.4 Product Expiry Alerts and Warning Configuration*

## 7.5 Product Catalog & Category Filtering

The Product Catalog (Figure 7.5) displays store merchandise categorized into distinct medical departments (Analgesic, Antibiotic, Antihistamine). Staff can quickly filter and search for items.

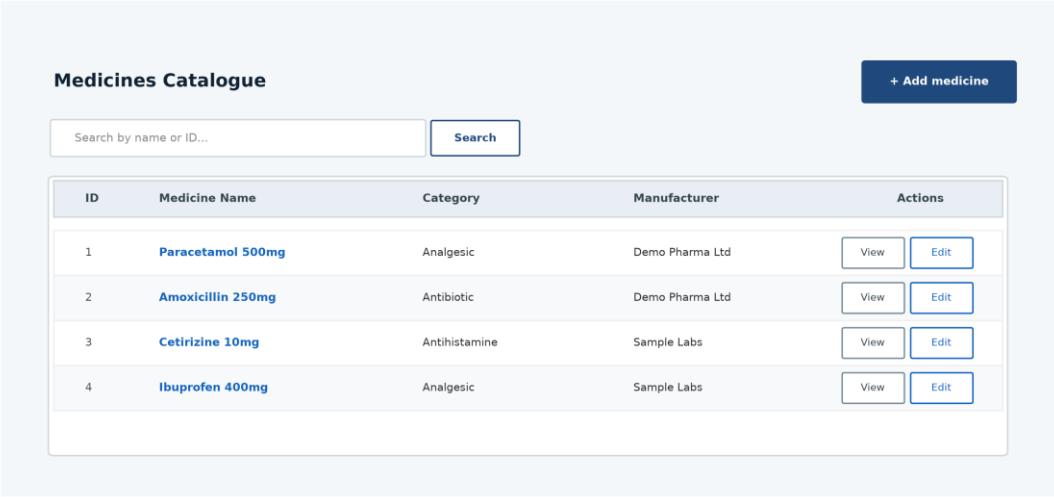



*Fig. 7.5 Product Catalog and Category View*

## 7.6 Add Product with Initial Batch Creation

The Add Product form (Figure 7.6) streamlines inventory intake. Creating a new product captures its name, category, manufacturer, and initial batch.

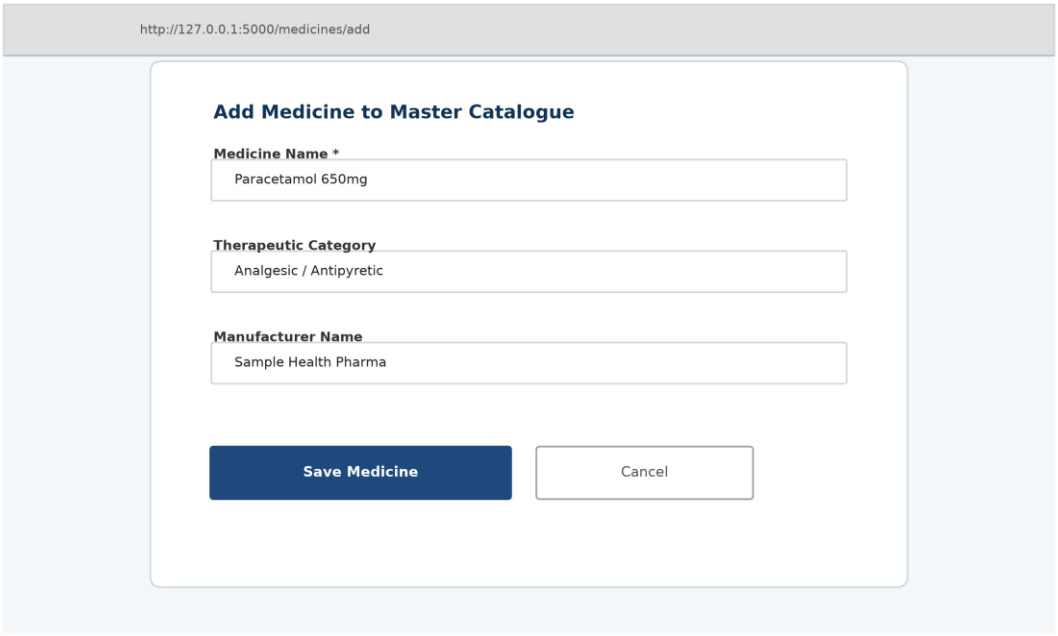



*Fig. 7.6 Add New Product with Initial Batch Creation*

## 7.7 FEFO Transaction Recording (Purchase & Sale)

The Transaction Entry screen (Figure 7.7) enables cashiers to record purchases and sales. When executing sales, the system automatically allocates deductions using the First-Expired, First-Out (FEFO) algorithm.

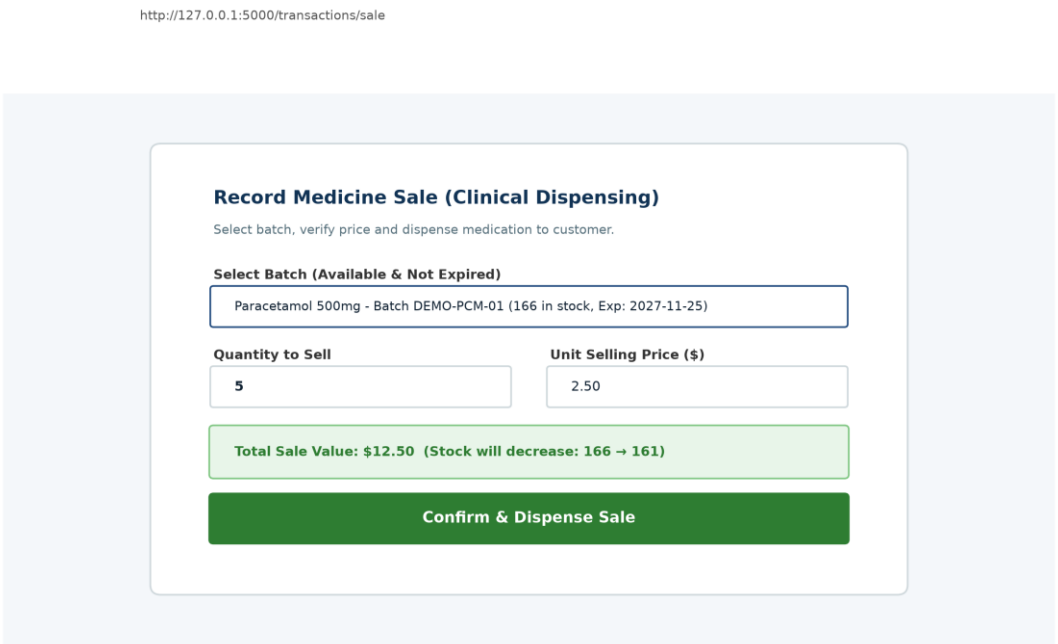


*Fig. 7.7 Transaction Entry (Purchase & FEFO Sale Processing)*

## 7.8 Transaction Audit Logs & Staff Traceability

The Transactions view (Figure 7.8) maintains an immutable audit trail of all store inventory movements. Each row records the exact timestamp, product, batch affected, quantity, and the acting staff member.

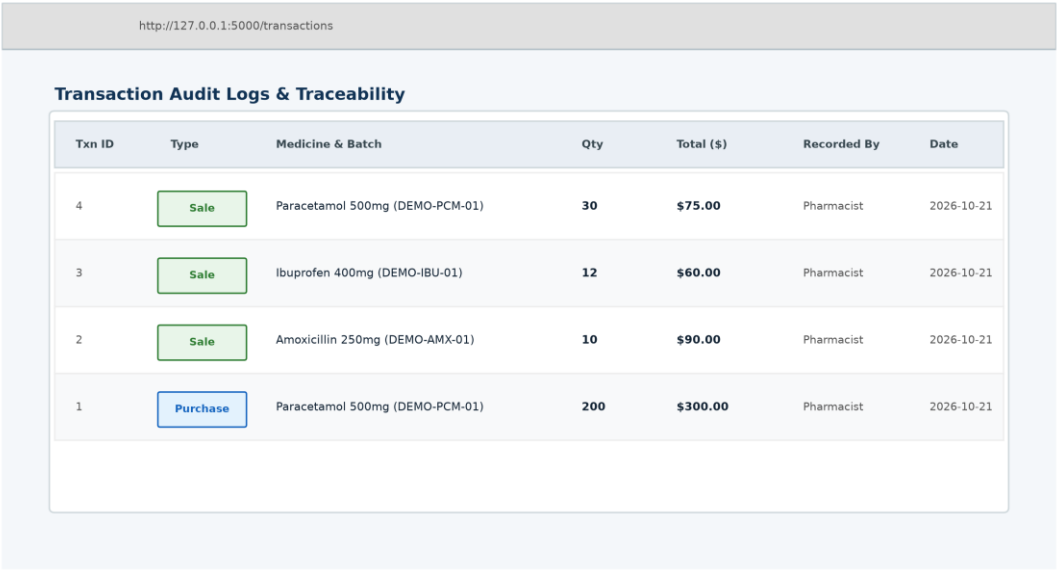



*Fig. 7.8 Transaction Audit Logs & Staff Traceability*

## 7.9 Operational Reporting & CSV Data Export

The Reports module (Figure 7.9) generates analytical summaries across operational categories, including expiring stock, low inventory, and Sales Count per Medicine, complete with one-click clean CSV export.

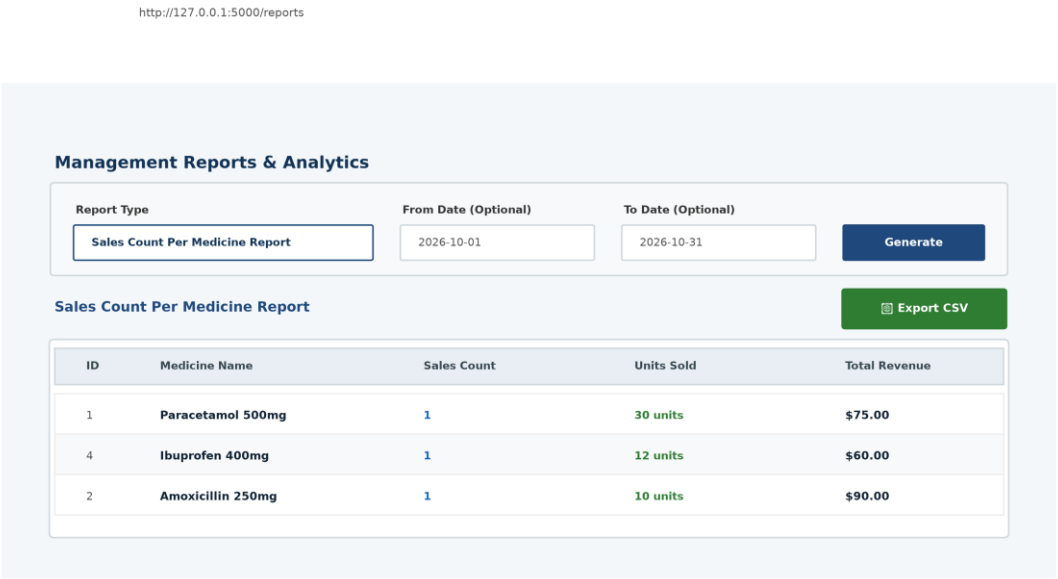


*Fig. 7.9 Store Reports & CSV Data Export*

## 7.10 Medicine Waste Management & Hazard Disposal

The Waste Management interface (Figure 7.10) empowers staff to record write-offs of expired or damaged drugs with formal justifications, preventing toxic or expired items from entering circulation.

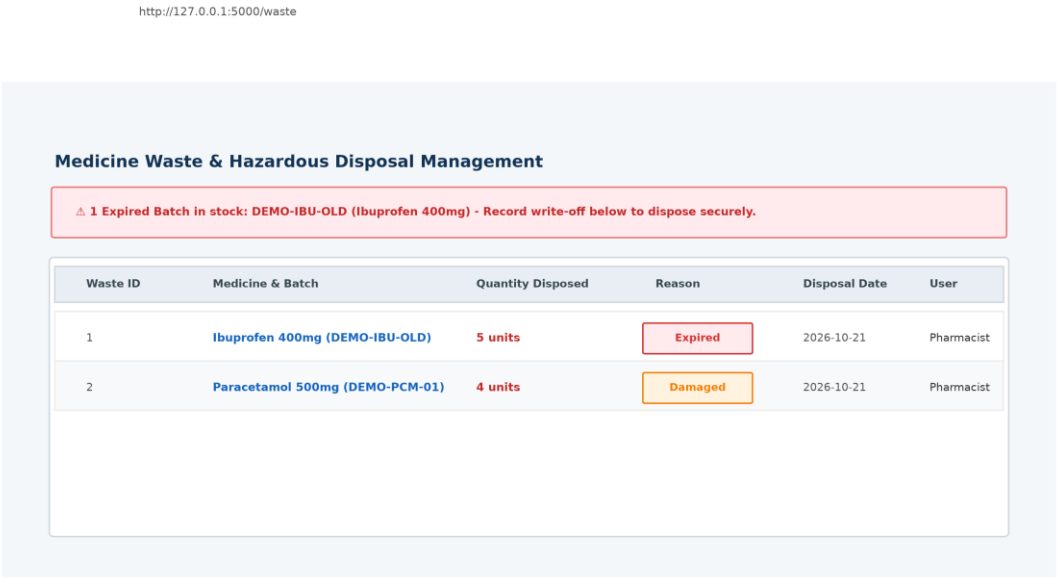



*Fig. 7.10 Medicine Waste Management & Hazard Disposal*

# CHAPTER 8: SAMPLE SOURCE CODE

This chapter presents key architectural source code implementations across all three tiers of the Smart Pharmacy Inventory and Medicine Waste Management System: the Database Layer (schema definition, foreign keys, CHECK constraints, and atomic transaction context manager), the Backend Application & Services Layer (First-Expired, First-Out FEFO sales deduction engine, dynamic shelf-life calculation, and role-based security middleware), and the Frontend Presentation Layer (Jinja2 point-of-sale dispensing templates and client-side reactive JavaScript form validation).

## 8.1 Database Architecture & Relational Schema (schema.sql & database.py)

The persistence layer is implemented using SQLite 3 with strict foreign key constraint enforcement and ACID transactional isolation. Figure 8.1 displays the relational schema definition in schema.sql, highlighting table structures for medicines, inventory batches, financial transactions, and hazard waste records, along with specialized compound unique constraints, CHECK constraints (e.g., quantity >= 0, purchase_price >= 0), and query-optimization indexes on expiry dates and timestamps.

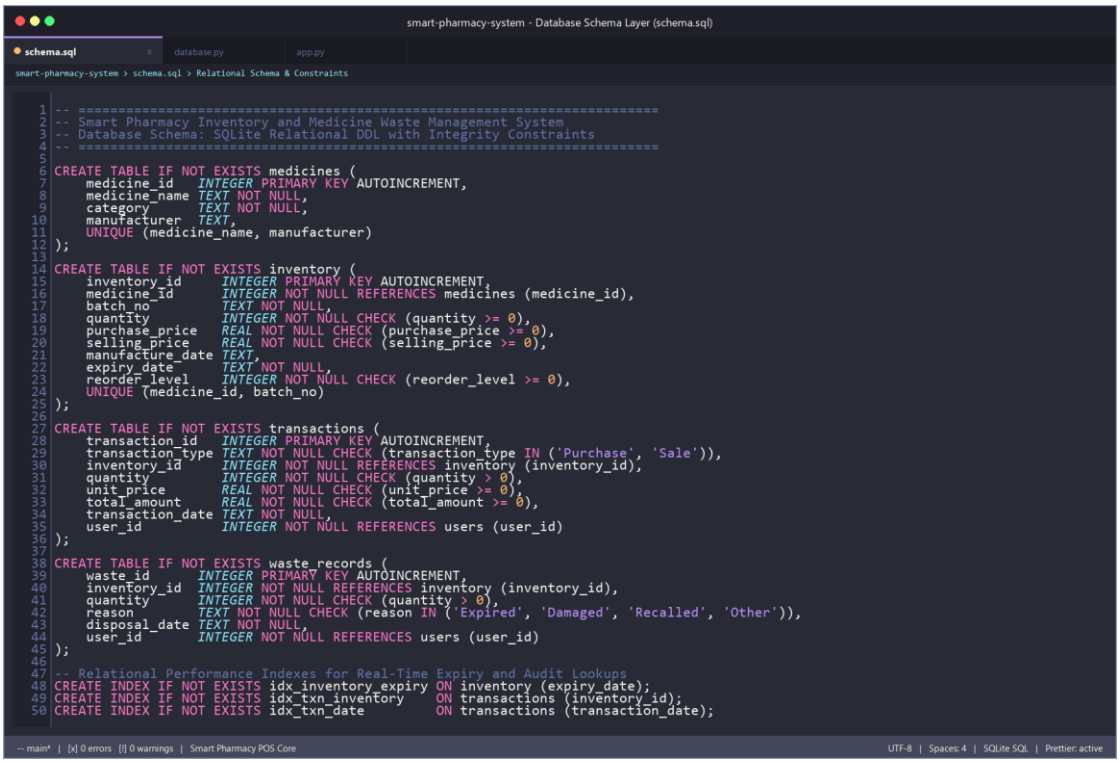



*Fig. 8.1 SQLite Relational Database Schema & Data Integrity Constraints (schema.sql)*

To guarantee atomic execution during concurrent inventory modifications and counter dispensing, database.py establishes connection pooling with PRAGMA foreign_keys = ON and encapsulates SQLite's BEGIN IMMEDIATE locking mechanism within a Python context manager class (Transaction). If an unexpected exception occurs mid-transaction (such as insufficient stock or expired batch validation failure), changes are immediately rolled back, preventing orphaned records or inventory drift, as shown in Figure 8.2.

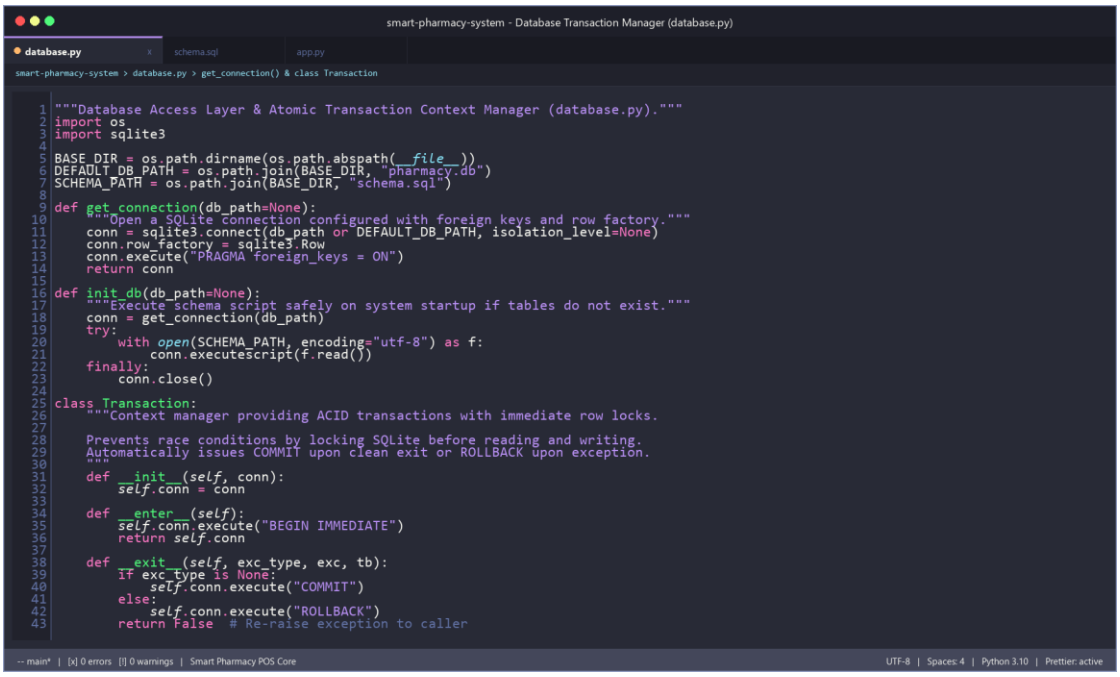



*Fig. 8.2 Database Connection Pool & Atomic Transaction Context Manager (database.py)*

## 8.2 Backend Core Services & FEFO Deduction Engine (transaction_service.py & inventory_service.py)

The core clinical dispensing workflow is governed by the First-Expired, First-Out (FEFO) sales deduction engine in services/transaction_service.py. When a dispensing transaction is initiated, the system re-reads the target batch under an immediate transaction lock, validates that the batch has not expired, checks that the available batch quantity satisfies the sale, decrements inventory stock atomically, and records an immutable transaction row linked to the active staff member's ID. Figure 8.3 illustrates this robust FEFO sales deduction algorithm.

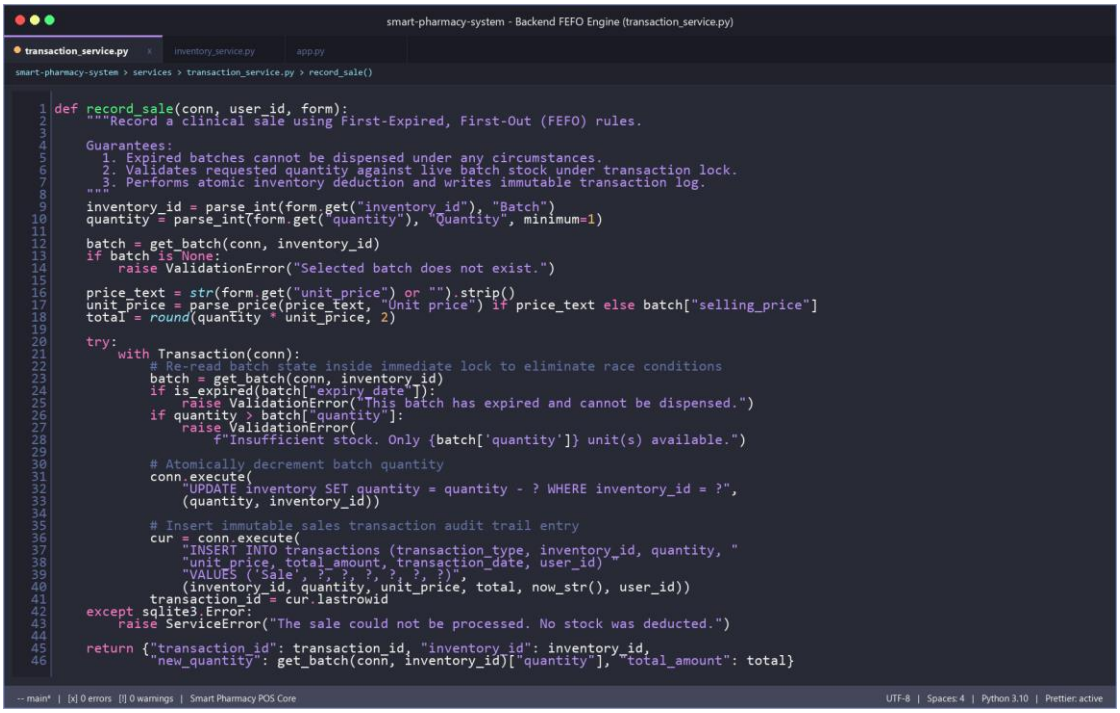



*Fig. 8.3 Backend First-Expired, First-Out (FEFO) Stock Allocation Engine (transaction_service.py)*

Complementing the sales engine, services/inventory_service.py implements the Dynamic Expiry Monitoring Engine. Rather than relying on asynchronous cron daemons that risk stale cached data, product shelf-life status (Available, Near Expiry within 30 days, Low Stock, Expired, and Out of Stock) is dynamically calculated relative to the system calendar at query execution time. Figure 8.4 illustrates this evaluation logic alongside the multi-batch stock aggregation query.

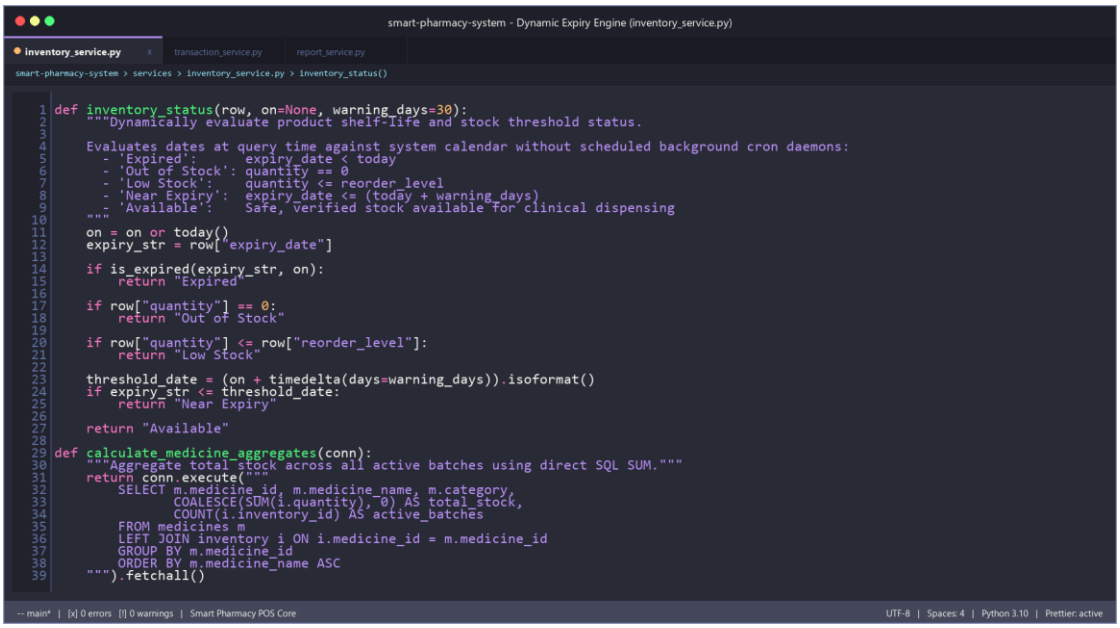



*Fig. 8.4 Backend Dynamic Shelf-Life Evaluation & Batch Stock Aggregation (inventory_service.py)*

## 8.3 Backend Application Controller & Security Middleware (app.py)

Application routing, HTTP dispatching, role-based access control (RBAC), and security middleware are centralized within app.py. The global @app.before_request hook authenticates user sessions and enforces cryptographic Hash-based Message Authentication Code (HMAC) anti-CSRF token verification on all state-altering POST requests. Furthermore, Figure 8.5 highlights the @login_required route decorator and the administrative Sales Count per Medicine analytics endpoint, which computes sales volumes, units sold, and revenue summaries.

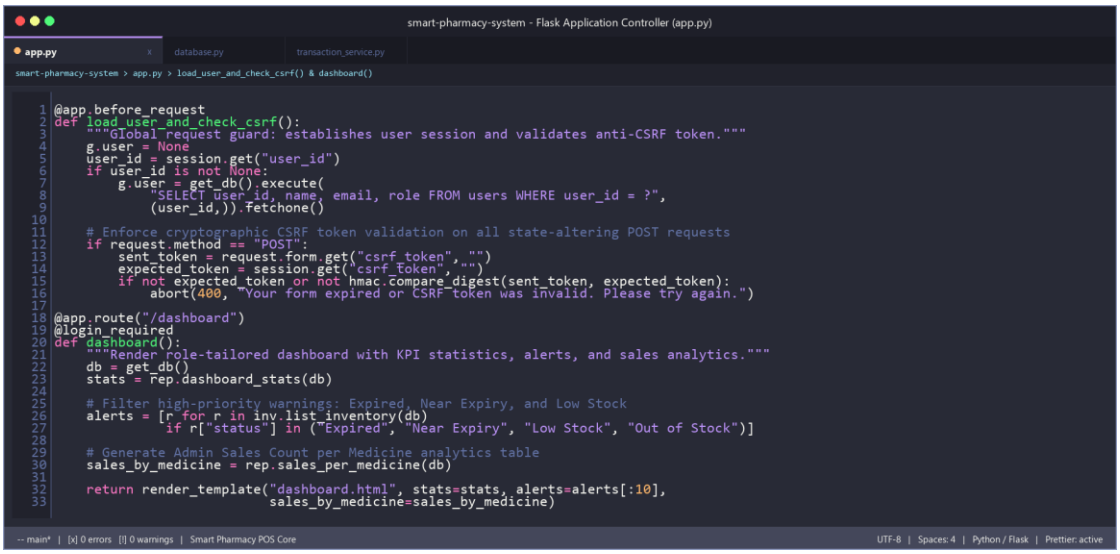



*Fig. 8.5 Flask Application Controller & Global CSRF Security Middleware (app.py)*

## 8.4 Frontend Point-of-Sale Dispensing Template (templates/transaction_form.html)

The presentation tier uses Jinja2 server-side templating combined with Bootstrap 5.3.3. Figure 8.6 displays the point-of-sale dispensing template (templates/transaction_form.html). The batch selection dropdown dynamically queries and renders only verified, non-expired inventory batches with real-time stock balances and expiry dates embedded directly within HTML data attributes (data-price, data-stock), providing cashier transparency while programmatically quarantining expired merchandise.

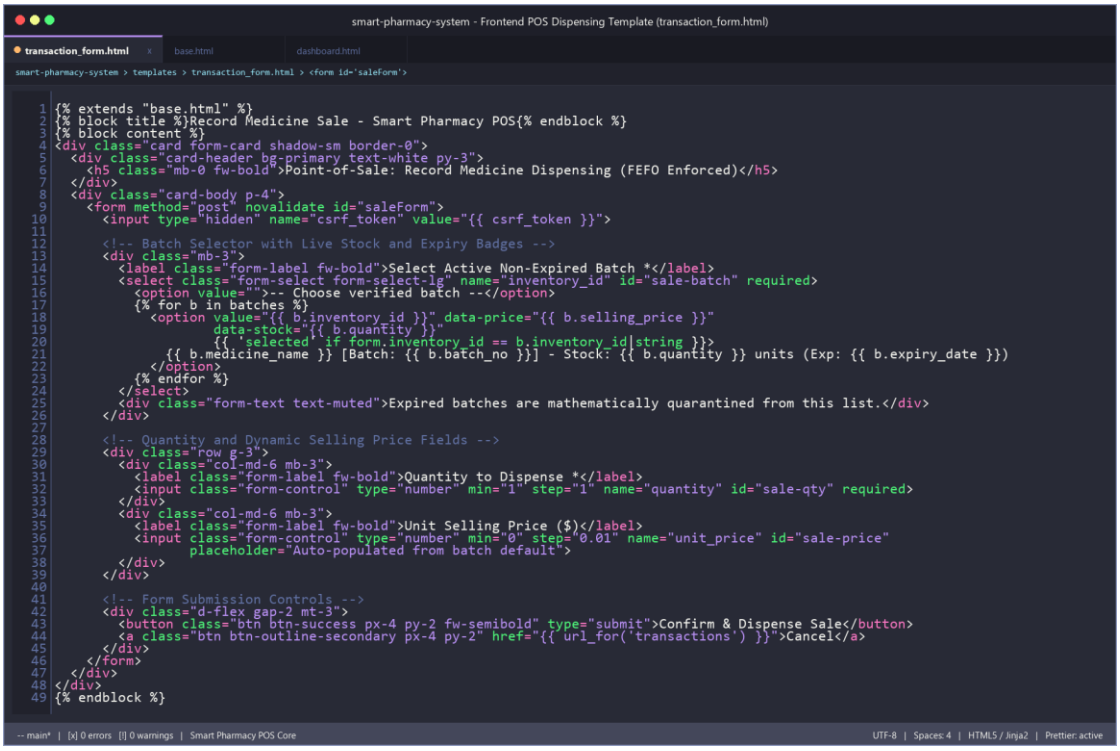



*Fig. 8.6 Frontend Point-of-Sale Dispensing Template with Dynamic Batch Selector (transaction_form.html)*

## 8.5 Frontend Client-Side Interactivity & Form Guard Logic (static/js/script.js)

To elevate user experience on pharmacy counter terminals without heavy single-page application frameworks, static/js/script.js provides lightweight, reactive client-side interactivity. Figure 8.7 illustrates the client-side listeners that dynamically auto-populate selling prices when a cashier selects a batch, highlight active navigation tabs matching current URL routes, and enforce an anti-double-submission lock that disables the submit button immediately upon execution to prevent accidental double-billing.

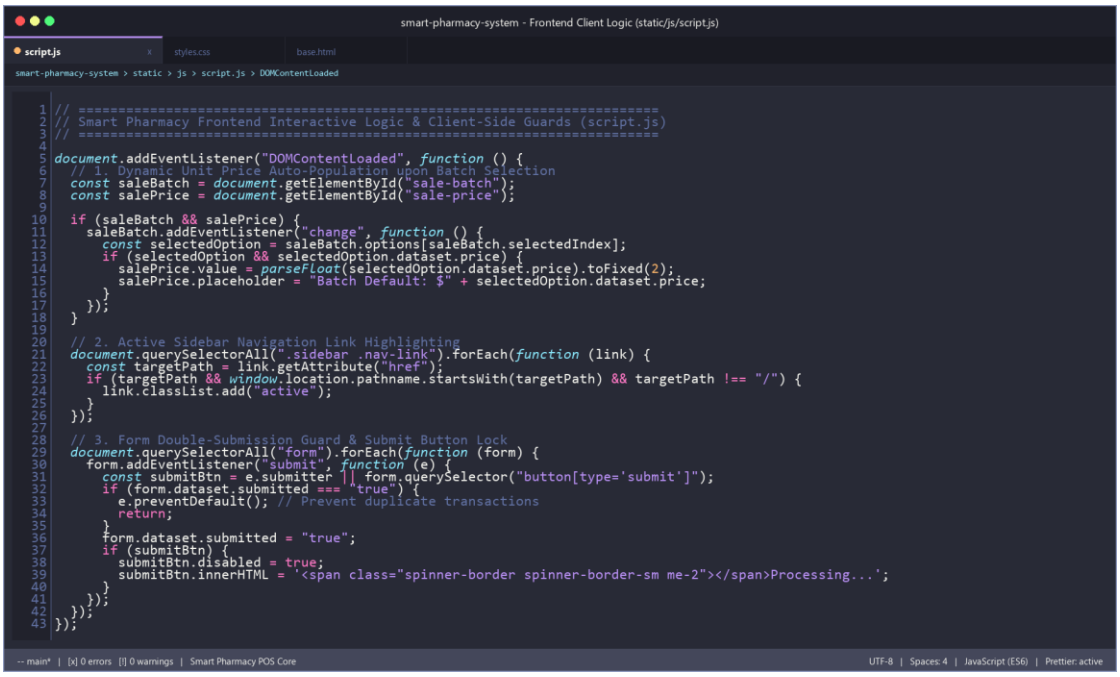



*Fig. 8.7 Frontend Client-Side Event Listeners & Double-Submit Guard (static/js/script.js)*

# CHAPTER 9: CONCLUSION & FUTURE ENHANCEMENTS

## 9.1 Conclusion

The Smart Pharmacy Inventory and Medicine Waste Management System successfully delivers a comprehensive, highly reliable, and accessible web-based operational solution tailored for small and medium retail stores and healthcare clinics. By replacing manual ledgers and unlinked spreadsheets with an automated, relational, batch-centric software architecture, the platform effectively resolves the primary operational challenges of the retail and pharmaceutical domain.

Key achievements of the developed project include:
• Elimination of Stock Discrepancies: By dynamically calculating total stock as the direct mathematical sum of active inventory batches, stock quantities remain 100% accurate without data drift.
• Automated FEFO Rotation: The First-Expired, First-Out sales allocation engine ensures older stock is prioritized for sales, significantly curbing inventory spoilage and perishable write-off costs.
• Proactive Consumer Safety: Expired stock is mathematically quarantined and blocked from point-of-sale execution, protecting customer health and upholding regulatory compliance.
• Zero Maintenance & High Portability: Operating seamlessly on standard Python and SQLite environments over local intranet connections, the system eliminates cloud hosting dependencies and expensive recurring license fees.

## 9.2 Future Enhancements

While the current platform fulfills all core operational requirements, future iterations can introduce several valuable capabilities:
• Hardware Barcode & QR Scanner Integration: Integrating native USB and Bluetooth barcode reader support to accelerate checkout counter operations.
• Supplier & Purchase Order Module: Enabling automated generation of re-order purchase requisitions sent directly to wholesale distributors when stock breaches minimum thresholds.
• Automated SMS & Email Notifications: Dispatching daily automated digest notifications to store managers alerting them to products reaching their warning threshold within 48 hours.
• Customer Receipt Billing & Thermal Printing: Integrating thermal receipt printer drivers to output itemized customer sales receipts upon checkout.

# CHAPTER 10: REFERENCES

1. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

3. Pallets Projects. (2026). Flask Documentation: The Python Microframework for Web Development. https://flask.palletsprojects.com/

4. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

5. World Wide Web Consortium (W3C). (2023). HTML5 & CSS3 Specification Guidelines. https://www.w3.org/TR/html5/

6. SQLite Consortium. (2026). SQLite In-Depth Architectural Documentation, ACID Transactions and SQL Syntax. https://www.sqlite.org/docs.html

7. Python Software Foundation. (2026). Python 3 Documentation: Built-in Data Structures and sqlite3 API. https://docs.python.org/3/

8. OWASP Foundation. (2025). Cross-Site Request Forgery (CSRF) & SQL Injection Prevention Cheat Sheets. https://cheatsheetseries.owasp.org/